

DecodeLabs Industrial Training Kit

Batch 2026

PROJECT 3

SQL Data Analysis

Querying for Truth — From Structured Queries to Cross-Validated Business Intelligence

[This report was prepared as part of the DecodeLabs Industrial Training Programme, Data Analytics track. It documents the SQL analysis of the dataset cleaned in Project 1 and analysed in Project 2, with every query result cross-validated against the Project 2 EDA figures.]

Dataset Analysed

E-Commerce Sales Dataset as— `dbo.Orders`, 1,200 rows × 14 columns

Environment: SQL Server Management Studio

Database: `Data_Analytics`

Submitted to DecodeLabs | July 2026

Executive Summary

Every SQL result in this project reconciles exactly with the Project 2 EDA figures — the same answers, produced independently in a second tool. That is the core outcome of this milestone: the analysis is not just repeated in SQL, it is verified by it.

The verified Project 1 dataset was imported into Microsoft SQL Server 2022 as `dbo.Orders` (1,200 rows × 14 columns) and interrogated through a six-query script covering the full requirement set of the brief: import verification with `COUNT`, a numeric precision correction with `UPDATE` and `ROUND`, row-level filtering with `WHERE`, ranking with `ORDER BY`, categorical aggregation with `GROUP BY` (`COUNT`, `SUM`, `AVG`), and aggregate-level filtering with `HAVING`.

The headline reconciliations: the row count (1,200), the number of Returned orders (247), the complete top-10 orders by value (led by `ORD200789` at £3,456.40), and the full revenue-by-product ranking (Chair £195,620.11 first, Phone £151,722.39 last) are all identical to the figures established in Excel in Project 2. Two tools, one truth — confirming both the integrity of the dataset and the correctness of the analysis on either side.

Table of Contents

Section 1: Methodology & Framework 1

 1.1 The Query Framework 1

 1.2 Environment & Data Source..... 1

 1.3 Key Techniques Applied 2

Section 2: Query 1 — Import Verification..... 3

Section 3: Query 2 — Numeric Precision Correction (UPDATE)..... 3

Section 4: Query 3 — Row-Level Filtering (WHERE)..... 5

Section 5: Query 4 — Ranking (ORDER BY) 6

Section 6: Query 5 — Aggregation (GROUP BY)..... 7

Section 7: Query 6 — Filtering Aggregates (HAVING)..... 8

Section 8: Cross-Validation Against Project 2 EDA..... 9

Section 9: Conclusion 9

Summary Results Table 10

Section 1: Methodology & Framework

1.1 The Query Framework

SQL is a declarative language: rather than writing step-by-step instructions, the analyst defines the desired output, and the database engine determines the most efficient way to fetch it. Each query in this project follows the Input–Process–Output cycle — a precise structured command (input), the engine's parsing, optimisation, and execution (process), and a clean, filtered result set (output).

The queries were designed around the engine's logical execution order rather than the written syntax order: FROM locates the data, WHERE filters individual rows, GROUP BY collapses rows into categorical buckets, HAVING filters those aggregated buckets, SELECT chooses and aliases the output columns, and ORDER BY sorts the final result. Respecting this sequence — filtering early, aggregating next, presenting last — is what allows the HAVING query in Section 7 to filter on an aggregate that no WHERE clause could touch, and explains why computed aliases such as Revenue can be referenced in ORDER BY but not in WHERE.

1.2 Environment & Data Source

Environment: Microsoft SQL Server 2022 (16.0 RTM), local instance, queried through SQL Server Management Studio (SSMS).

Data source: the cleaned, verified dataset from Project 1 — 1,200 e-commerce order records × 14 columns — imported into the Data_Analytics database as the table dbo.Orders. Because the dataset passed the Project 1 verification gate (zero duplicate IDs, zero invalid dates, zero blanks, zero calculation mismatches), any result produced here can be compared like-for-like against the Project 2 EDA figures.

Execution evidence: every query in this report is shown executing in SSMS, with its result grid, in Figures 1–7. The status bar in Figure 1 confirms the environment and the full script's combined output of 282 rows (1 + 10 + 247 + 10 + 7 + 7 across the six queries).

```

1  --Verify the import
2  SELECT COUNT(*) FROM dbo.Orders;
3  SELECT TOP 10 * FROM dbo.Orders;
4
5  --Fix data types/rounding
6  UPDATE dbo.Orders
7  SET UnitPrice= ROUND(UnitPrice, 2),
8      TotalPrice= ROUND(TotalPrice, 2);
9
10 --Basic queries with Condition
11 SELECT OrderID, Product, TotalPrice
12 FROM dbo.Orders
13 WHERE OrderStatus = 'Returned';
14
15 --Top 10 Orders by Value
16 SELECT TOP 10 OrderID, Product, Quantity, TotalPrice
17 FROM dbo.Orders
18 ORDER BY TotalPrice DESC;
19

```

100 % No issues found Ln: 12 Ch: 16 TABS CRLF

Results Messages

(No column name)

1 1200

OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice	
1	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK37947903	7	SAVE10	Instagram	2853.1
2	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91186779	3	SAVE10	Referral	302.7
3	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42903982	8	FREESHIP	Email	2753.4
4	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Debit Card	Returned	TRK62788070	5	SAVE10	Facebook	273.19
5	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	Email	2504.04
6	ORD200005	2023-10-23	C37249	Phone	2	245.86	934 Main St	Credit Card	Shipped	TRK72976927	4	SAVE10	Instagram	491.72
7	ORD200006	2025-06-17	C83492	Laptop	1	664.42	986 Main St	Gift Card	Returned	TRK96417362	6	SAVE10	Facebook	664.42
8	ORD200007	2023-05-12	C41460	Monitor	5	149.55	706 Main St	Cash	Shipped	TRK78809193	9	FREESHIP	Facebook	747.75

OrderID	Product	TotalPrice	
163	ORD200753	Phone	482.98
164	ORD200754	Printer	904.1
165	ORD200762	Phone	230.88
166	ORD200765	Laptop	1069.82
167	ORD200770	Desk	314.38
168	ORD200776	Tablet	21.19
169	ORD200782	Monitor	987.26
170	ORD200783	Printer	438.34

Query executed successfully. RUBEL\MSSQLSERVER01 (16.0 RTM) RUBEL\User (75) Data_Analytics 00:00:00 282 rows

Figure 1: The full script in SSMS — connected to RUBEL\MSSQLSERVER01, database Data_Analytics; 'Query executed successfully — 282 rows'.

1.3 Key Techniques Applied

- **SELECT:** column selection and aliasing of computed metrics (AS Orders, AS Revenue, AS AvgOrder).
- **WHERE:** row-level filtering on order status.
- **ORDER BY with TOP:** ranking orders by value, sorting on a computed alias, and TOP N result limiting.
- **GROUP BY:** collapsing 1,200 rows into 7 product buckets.
- **Aggregate functions:** COUNT (*) for volume, SUM for financial totals, AVG for typicality — the three executive aggregations.
- **HAVING:** filtering on an aggregate after grouping — the clause WHERE cannot replicate.
- **UPDATE:** in-database data correction with ROUND, mirroring the Project 1 fix at the database layer.

Section 2: Query 1 — Import Verification

Before any analysis, the import itself was verified — the SQL equivalent of Project 1's audit-first discipline. Two checks: a full row count, and a TOP 10 sample to inspect column alignment and data types by eye.

```
--Verify the import
SELECT COUNT (*) FROM dbo.Orders;
SELECT TOP 10 * FROM dbo.Orders;
```

COUNT (*) returned exactly 1,200 — every record from the verified Excel dataset arrived in dbo.Orders, with none lost or duplicated in transit. COUNT (*) was chosen deliberately: it tallies every row including NULLs, so it verifies physical completeness rather than the completeness of any single column. The TOP 10 sample confirmed all 14 columns imported with correct headers, readable dates, and sensible values (e.g. ORD200000, 2023-01-04, Monitor, £2,853.10).

The screenshot shows a SQL query window with the following text:

```
--Verify the import
SELECT COUNT(*) FROM dbo.Orders;
SELECT TOP 10 * FROM dbo.Orders;
```

Below the query window, the 'Results' tab is active, showing a table with 14 columns and 10 rows of data. The first row is highlighted.

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice
1	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK37947903	7	SAVE10	Instagram	2853.1
2	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91186779	3	SAVE10	Referral	302.7
3	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42903982	8	FREESHIP	Email	2753.4
4	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Debit Card	Returned	TRK62788070	5	SAVE10	Facebook	273.19
5	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	Email	2504.04
6	ORD200005	2023-10-23	C37249	Phone	2	245.86	934 Main St	Credit Card	Shipped	TRK72976927	4	SAVE10	Instagram	491.72
7	ORD200006	2025-06-17	C83492	Laptop	1	664.42	986 Main St	Gift Card	Returned	TRK96417362	6	SAVE10	Facebook	664.42
8	ORD200007	2023-05-12	C41460	Monitor	5	149.55	706 Main St	Cash	Shipped	TRK78809193	9	FREESHIP	Facebook	747.75
9	ORD200008	2025-04-02	C26817	Phone	2	134.28	904 Main St	Gift Card	Cancelled	TRK61042692	2	NO_COUPON	Email	268.56
10	ORD200009	2023-11-21	C31946	Desk	4	509.38	102 Main St	Credit Card	Shipped	TRK33478363	6	SAVE10	Google	2037.52

Figure 2: Import verification — COUNT (*) = 1,200 and the TOP 10 sample with all 14 columns intact.

Section 3: Query 2 — Numeric Precision Correction (UPDATE)

Inspection of the imported prices showed floating-point representation artefacts had re-appeared at the database layer — the import maps Excel numerics to floating-point storage, re-introducing the same class of issue Project 1 corrected in the workbook. The fix was applied in-database:

```
--Fix data types/rounding
UPDATE dbo.Orders
SET UnitPrice= ROUND(UnitPrice, 2),
    TotalPrice= ROUND(TotalPrice, 2);
```

The UPDATE affected all 1,200 rows, setting both price columns to exact 2-decimal currency precision. This repeats Project 1's CR004/CR005 correction at the storage layer — a practical demonstration that data cleaning is not a one-time event but a discipline re-applied at every hand-off between tools. A follow-up SELECT confirmed the corrected values (e.g. ORD200011 at 147.42, ORD200019 at 1120).

1	--Fix data types/rounding
2	UPDATE dbo.Orders
3	SET UnitPrice= ROUND(UnitPrice, 2),
4	TotalPrice= ROUND(TotalPrice, 2);
5	SELECT * FROM dbo.Orders

00 %

No issues found

Ln: 5

Ch

OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice
ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK37947903	7	SAVE10	Instagram	2853.1
ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91186779	3	SAVE10	Referral	302.7
ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42903982	8	FREESHIP	Email	2753.4
ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Debit Card	Returned	TRK62788070	5	SAVE10	Facebook	273.19
ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	Email	2504.04
ORD200005	2023-10-23	C37249	Phone	2	245.86	934 Main St	Credit Card	Shipped	TRK72976927	4	SAVE10	Instagram	491.72
ORD200006	2025-06-17	C83492	Laptop	1	664.42	986 Main St	Gift Card	Returned	TRK96417362	6	SAVE10	Facebook	664.42
ORD200007	2023-05-12	C41460	Monitor	5	149.55	706 Main St	Cash	Shipped	TRK78809193	9	FREESHIP	Facebook	747.75
ORD200008	2025-04-02	C26817	Phone	2	134.28	904 Main St	Gift Card	Cancelled	TRK61042692	2	NO_COUPON	Email	268.56
ORD200009	2023-11-21	C31946	Desk	4	509.38	102 Main St	Credit Card	Shipped	TRK33478363	6	SAVE10	Google	2037.52
ORD200010	2023-12-29	C43443	Tablet	5	625.97	333 Main St	Credit Card	Returned	TRK98859248	9	WINTER15	Instagram	3129.85
ORD200011	2024-02-17	C93861	Monitor	3	49.14	831 Main St	Online	Returned	TRK48234646	7	SAVE10	Email	147.42
ORD200012	2024-10-15	C38785	Monitor	2	180.5	179 Main St	Debit Card	Pending	TRK20419991	6	FREESHIP	Referral	361
ORD200013	2023-08-30	C88348	Laptop	3	201.49	903 Main St	Credit Card	Returned	TRK81913874	4	FREESHIP	Facebook	604.47
ORD200014	2023-03-27	C98474	Tablet	2	393.33	980 Main St	Debit Card	Pending	TRK79186539	3	SAVE10	Instagram	786.66
ORD200015	2023-07-17	C39416	Printer	1	473.96	942 Main St	Cash	Delivered	TRK54930938	3	NO_COUPON	Google	473.96
ORD200016	2023-01-24	C28976	Printer	2	533.81	914 Main St	Credit Card	Pending	TRK13194259	3	FREESHIP	Email	1067.62
ORD200017	2024-03-02	C60672	Tablet	1	423.4	382 Main St	Debit Card	Shipped	TRK11972116	3	NO_COUPON	Referral	423.4
ORD200018	2025-03-26	C84329	Desk	1	431.44	891 Main St	Gift Card	Pending	TRK65191045	3	SAVE10	Facebook	431.44
ORD200019	2023-01-23	C14552	Monitor	5	224	273 Main St	Cash	Returned	TRK59101365	9	SAVE10	Google	1120

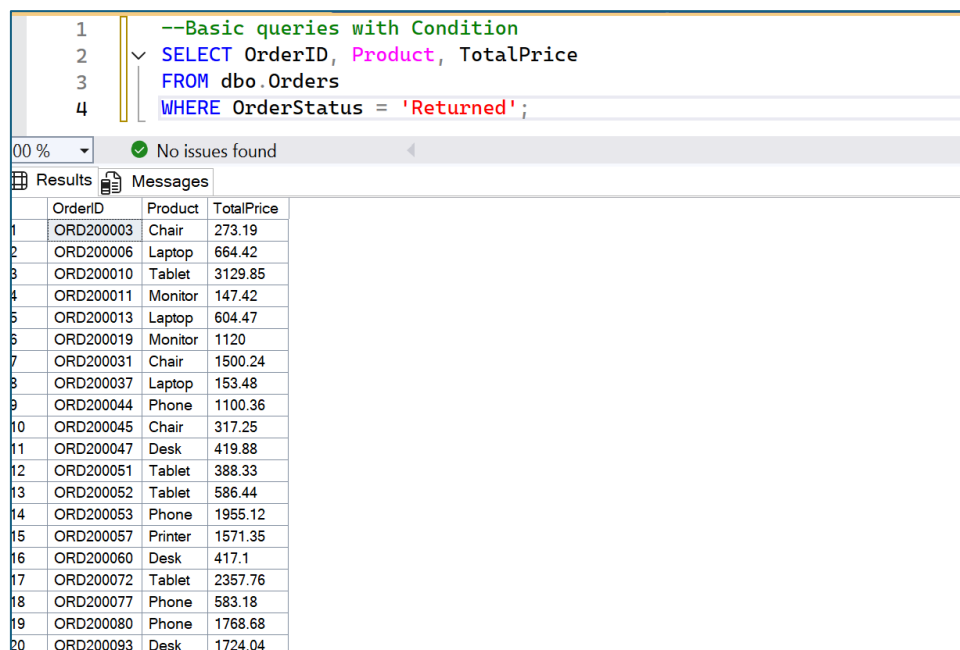
Figure 3: The UPDATE ... ROUND correction and the follow-up SELECT showing 2-decimal prices.

Section 4: Query 3 — Row-Level Filtering (WHERE)

The WHERE clause acts as a strict row-level funnel: it evaluates every individual row and discards what does not match. Here it isolates the Returned orders — one half of the fulfilment-risk segment identified as the headline finding of Project 2.

```
--Basic queries with Condition
SELECT OrderID, Product, TotalPrice
FROM dbo.Orders
WHERE OrderStatus = 'Returned';
```

The query returned 247 rows — exactly matching the Returned count in the Project 2 order-status breakdown. Sample results include ORD200003 (Chair, £273.19), ORD200010 (Tablet, £3,129.85), and ORD200093 (Desk, £1,724.04). This filtered set is the query-level foundation for any future returns investigation: the equality filter can be extended with comparison operators (e.g. TotalPrice thresholds) or pattern matching as the diagnosis narrows.



	OrderID	Product	TotalPrice
1	ORD200003	Chair	273.19
2	ORD200006	Laptop	664.42
3	ORD200010	Tablet	3129.85
4	ORD200011	Monitor	147.42
5	ORD200013	Laptop	604.47
6	ORD200019	Monitor	1120
7	ORD200031	Chair	1500.24
8	ORD200037	Laptop	153.48
9	ORD200044	Phone	1100.36
10	ORD200045	Chair	317.25
11	ORD200047	Desk	419.88
12	ORD200051	Tablet	388.33
13	ORD200052	Tablet	586.44
14	ORD200053	Phone	1955.12
15	ORD200057	Printer	1571.35
16	ORD200060	Desk	417.1
17	ORD200072	Tablet	2357.76
18	ORD200077	Phone	583.18
19	ORD200080	Phone	1768.68
20	ORD200093	Desk	1724.04

Figure 4: WHERE OrderStatus = 'Returned' — the filtered result set (247 rows).

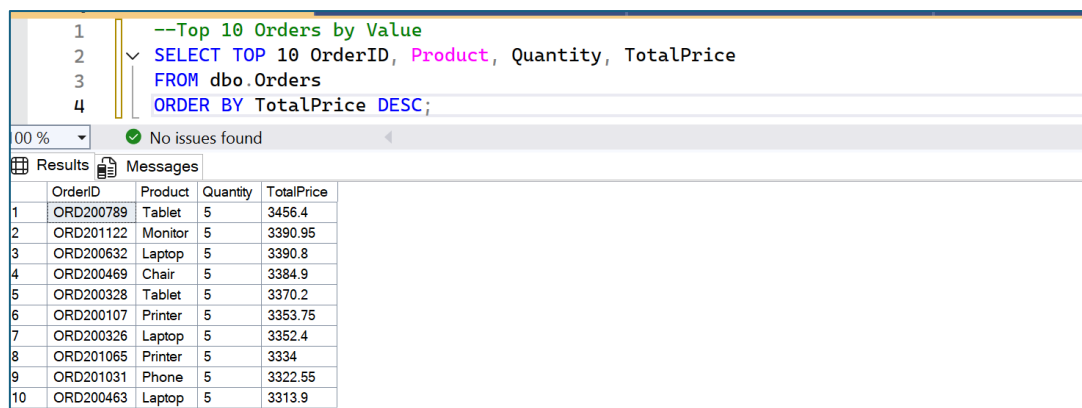
Section 5: Query 4 — Ranking (ORDER BY)

ORDER BY operates at the end of the execution order, sorting the final result set; combined with TOP it produces a ranking. Here it surfaces the ten highest-value orders:

```
--Top 10 Orders by Value
SELECT TOP 10 OrderID, Product, Quantity, TotalPrice
FROM dbo.Orders
ORDER BY TotalPrice DESC;
```

Rank	OrderID	Product	Quantity	TotalPrice (£)
1	ORD200789	Tablet	5	3,456.40
2	ORD201122	Monitor	5	3,390.95
3	ORD200632	Laptop	5	3,390.80
4	ORD200469	Chair	5	3,384.90
5	ORD200328	Tablet	5	3,370.20
6	ORD200107	Printer	5	3,353.75
7	ORD200326	Laptop	5	3,352.40
8	ORD201065	Printer	5	3,334.00
9	ORD201031	Phone	5	3,322.55
10	ORD200463	Laptop	5	3,313.90

This top-10 list is identical, order for order, to the outlier inspection list produced by the Project 2 IQR analysis — and it independently confirms that analysis's verdict: every one of these orders is a maximum-quantity (5-unit) purchase of a high-priced product. Legitimate premium demand, not data error.



The screenshot shows a SQL query window with the following text:

```
--Top 10 Orders by Value
SELECT TOP 10 OrderID, Product, Quantity, TotalPrice
FROM dbo.Orders
ORDER BY TotalPrice DESC;
```

Below the query window, the 'Results' tab is active, displaying a grid with the following data:

	OrderID	Product	Quantity	TotalPrice
1	ORD200789	Tablet	5	3456.4
2	ORD201122	Monitor	5	3390.95
3	ORD200632	Laptop	5	3390.8
4	ORD200469	Chair	5	3384.9
5	ORD200328	Tablet	5	3370.2
6	ORD200107	Printer	5	3353.75
7	ORD200326	Laptop	5	3352.4
8	ORD201065	Printer	5	3334
9	ORD201031	Phone	5	3322.55
10	ORD200463	Laptop	5	3313.9

Figure 5: TOP 10 ... ORDER BY TotalPrice DESC — execution and result grid.

Section 6: Query 5 — Aggregation (GROUP BY)

GROUP BY collapses the 1,200 individual rows into 7 product buckets, and the three executive aggregations summarise each bucket: COUNT (*) for volume, SUM for financial scale, AVG for typicality. ORDER BY then sorts on the computed alias Revenue — legal precisely because ORDER BY executes after SELECT has created the alias.

```
--Revenue by Product
SELECT Product, COUNT (*) AS Orders, SUM(TotalPrice) AS Revenue, AVG(TotalPrice) AS AvgOrder
FROM dbo.Orders
GROUP BY Product
ORDER BY Revenue DESC;
```

Product	Orders	Revenue (£)	AvgOrder (£)
Chair	178	195,620.11	1,098.99
Printer	181	195,612.61	1,080.73
Laptop	173	192,126.56	1,110.56
Tablet	179	186,568.95	1,042.28
Monitor	163	175,651.41	1,077.62
Desk	170	167,459.93	985.06
Phone	156	151,722.39	972.58

Every figure — order counts, revenue totals, and average order values — matches the Project 2 pivot-table breakdown exactly, down to the penny. The ranking retells the EDA story from the database side: Chair narrowly leads Printer (£7.50 apart across ~£195K), Phone trails, and the whole range spans only 12–15.5% of total revenue — an evenly spread portfolio.

Product	Orders	Revenue	AvgOrder
1 Chair	178	195620.11	1098.98938202247
2 Printer	181	195612.61	1080.7326519337
3 Laptop	173	192126.56	1110.55815028902
4 Tablet	179	186568.95	1042.28463687151
5 Monitor	163	175651.41	1077.61601226994
6 Desk	170	167459.93	985.058411764706
7 Phone	156	151722.39	972.579423076923

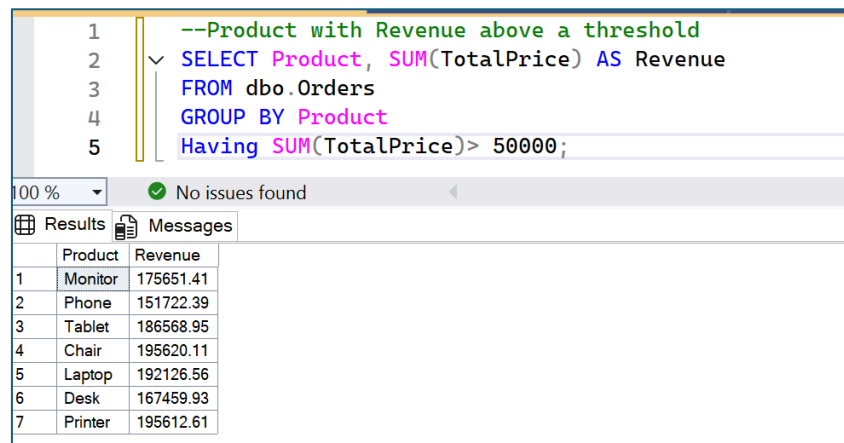
Figure 6: GROUP BY Product with COUNT, SUM, and AVG — note the full-precision AVG values later rounded for presentation.

Section 7: Query 6 — Filtering Aggregates (HAVING)

The final query demonstrates the clause that separates grouped analysis from row analysis. A revenue threshold cannot be applied with WHERE, because WHERE executes before GROUP BY — at that stage, SUM(TotalPrice) does not yet exist. HAVING executes after grouping, so it filters the aggregated buckets themselves:

```
--Product with Revenue above a threshold
SELECT Product, SUM(TotalPrice) AS Revenue
FROM dbo.Orders
GROUP BY Product
Having SUM(TotalPrice) > 50000;
```

All 7 products cleared the £50,000 threshold — consistent with the even revenue spread: the weakest category (Phone, £151,722) sits more than three times above the bar. The query pattern matters beyond this result: raising the threshold (e.g. to £190,000, which only Chair and Printer would pass) turns it into a top-performer filter, and the same HAVING structure underpins exception reporting of every kind.



The screenshot shows a SQL query editor with the following code:

```
1  --Product with Revenue above a threshold
2  SELECT Product, SUM(TotalPrice) AS Revenue
3  FROM dbo.Orders
4  GROUP BY Product
5  Having SUM(TotalPrice) > 50000;
```

Below the query editor, the 'Results' tab is active, displaying a table with 7 rows and 2 columns: Product and Revenue.

	Product	Revenue
1	Monitor	175651.41
2	Phone	151722.39
3	Tablet	186568.95
4	Chair	195620.11
5	Laptop	192126.56
6	Desk	167459.93
7	Printer	195612.61

Figure 7: HAVING SUM(TotalPrice) > 50000 — all seven product buckets pass the aggregate filter.

Section 8: Cross-Validation Against Project 2 EDA

The defining feature of this project is that its results were not merely produced — they were reconciled. Each SQL output was compared against the corresponding figure from the Project 2 Excel analysis:

Check	SQL Result	Project 2 EDA	Match
Total records	1,200	1,200	✓
Returned orders	247	247	✓
Highest-value order	ORD200789 · £3,456.40	ORD200789 · £3,456.40	✓
Top-10 orders by value	Identical list, ranks 1–10	IQR outlier inspection list	✓
Top product by revenue	Chair · £195,620.11	Chair · £195,620.11	✓
Lowest product by revenue	Phone · £151,722.39	Phone · £151,722.39	✓
Full product ranking (7 rows)	Identical order & figures	Identical order & figures	✓
Products above £50K revenue	7 of 7	7 of 7 (implied)	✓

A 100% reconciliation rate across two independent tools is strong evidence on two fronts at once: the dataset survived the Excel-to-SQL hand-off intact, and the analytical logic — pivot tables on one side, GROUP BY aggregations on the other — was implemented correctly on both. Any discrepancy in this table would have pointed to an import fault or a formula error; there were none.

Section 9: Conclusion

Project 3 bridged the gap between spreadsheet analysis and database querying. The six-query script covered every requirement of the brief — SELECT, WHERE, ORDER BY, GROUP BY, the COUNT/SUM/AVG aggregations, and HAVING — while re-deriving the key findings of Project 2 through pure relational logic: the 247-order returns segment, the ten legitimate premium orders, and the evenly spread product portfolio led by Chair and trailed by Phone.

The discipline established in Projects 1 and 2 carried through: verify before analysing (Query 1), re-apply data corrections at every tool hand-off (Query 2), and never accept a result that has not been reconciled against an independent source (Section 8). The database layer built here — a verified dbo.Orders table whose aggregations are proven to match the EDA — becomes the trusted foundation for Project 4, where these same figures are amplified through an interactive Power BI dashboard.

Summary Results Table

The table below consolidates every key metric from this project into a single reference.

Metric	Result
Environment	SQL Server 2022 (16.0) · SSMS · Data_Analytics database
Table analysed	dbo.Orders — 1,200 rows × 14 columns
Queries in scope	6 (verification, UPDATE, WHERE, ORDER BY, GROUP BY, HAVING)
Import verification (COUNT)	1,200 rows — matches source exactly
Rows corrected by UPDATE/ROUND	1,200 (UnitPrice and TotalPrice to 2 dp)
Returned orders (WHERE)	247 — matches EDA status mix
Highest-value order (ORDER BY)	ORD200789 · Tablet · £3,456.40
Top product by revenue (GROUP BY)	Chair — £195,620.11 (178 orders, AvgOrder £1,098.99)
Lowest product by revenue	Phone — £151,722.39
Products above £50K threshold (HAVING)	7 of 7
Cross-validation vs Project 2 EDA	8 of 8 checks reconciled — 100%
Artifacts	.sql script + SSMS execution screenshots

Analyst: Rubel Hasan · DecodeLabs Industrial Training Programme · Data Analytics Track · Batch 2026. This report was prepared as part of a training programme; the dataset was provided by DecodeLabs for educational purposes.